

Árvore Binária de Pesquisa

Estrutura da Árvore

```
package Arvore.Arvore_binaria;

public class Arvore_binariaNo implements Position {

    private No root;
    private int size;

    public Arvore_binariaNo() {
        this.root = null;
        this.size = 0;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public No getRoot() throws InvalidPositionExceptionBinaria {

        if (isEmpty()) {
            throw new InvalidPositionExceptionBinaria("Árvore vazia");
        }

        return root;
    }

    public No parent(No node) throws InvalidPositionExceptionBinaria {

        if (node == null) {
            throw new InvalidPositionExceptionBinaria("Nó nulo");
        }

        return node.getParent();
    }

    public No leftChild(No node) throws InvalidPositionExceptionBinaria {

        if (node == null) {
            throw new InvalidPositionExceptionBinaria("Nó nulo");
        }
    }
}
```

```

        return node.getChildrenEsq();
    }

    public No rightChild(No node) throws InvalidPositionExceptionBinaria {

        if (node == null) {
            throw new InvalidPositionExceptionBinaria("Nó nulo");
        }

        return node.getChildrenDir();
    }

    public boolean hasLeft(No node) throws InvalidPositionExceptionBinaria {

        if (node == null) {
            throw new InvalidPositionExceptionBinaria("Nó nulo");
        }

        if (node.getChildrenEsq() != null) {
            return true;
        }

        return false;
    }

    public boolean hasRight(No node) throws InvalidPositionExceptionBinaria {

        if (node == null) {
            throw new InvalidPositionExceptionBinaria("Nó nulo");
        }

        if (node.getChildrenDir() != null) {
            return true;
        }

        return false;
    }

    public boolean isRoot(No node) throws InvalidPositionExceptionBinaria {

        if (node == null) {
            throw new InvalidPositionExceptionBinaria("Nó nulo");
        }

        if (node == root) {
            return true;
        }

        return false;
    }
}

```

```

public boolean isInternal(No node) throws InvalidPositionExceptionBinaria {

    if (node == null) {
        throw new InvalidPositionExceptionBinaria("Nó nulo");
    }

    if (node.getChildrenEsq() != null || node.getChildrenDir() != null) {
        return true;
    }

    return false;
}

public boolean isExternal(No node) throws InvalidPositionExceptionBinaria {

    if (node == null) {
        throw new InvalidPositionExceptionBinaria("Nó nulo");
    }

    if (node.getChildrenEsq() == null && node.getChildrenDir() == null) {
        return true;
    }

    return false;
}

public boolean isLeftChild(No node) throws InvalidPositionExceptionBinaria {

    if (node == null) {
        throw new InvalidPositionExceptionBinaria("Nó nulo");
    }

    No pai = node.getParent();

    if (pai == null) {
        return false;
    }

    if (pai.getChildrenEsq() == node) {
        return true;
    }

    return false;
}

public boolean isRightChild(No node) throws InvalidPositionExceptionBinaria {

    if (node == null) {
        throw new InvalidPositionExceptionBinaria("Nó nulo");
    }

```

```

    }

    No pai = node.getParent();

    if (pai == null) {
        return false;
    }

    if (pai.getChildrenDir() == node) {
        return true;
    }

    return false;
}

public int height(No node) throws InvalidPositionExceptionBinaria {

    if (node == null) {
        throw new InvalidPositionExceptionBinaria("Nó nulo");
    }

    if (node.getChildrenEsq() == null && node.getChildrenDir() == null) {
        return 0;
    }

    int alturaEsq = 0;
    int alturaDir = 0;

    if (node.getChildrenEsq() != null) {
        alturaEsq = 1 + height(node.getChildrenEsq());
    }

    if (node.getChildrenDir() != null) {
        alturaDir = 1 + height(node.getChildrenDir());
    }

    if (alturaEsq > alturaDir) {
        return alturaEsq;
    }

    return alturaDir;
}

public int depth(No node) throws InvalidPositionExceptionBinaria {

    if (node == null) {
        throw new InvalidPositionExceptionBinaria("Nó nulo");
    }

    int profundidade = 0;
    No atual = node;

```

```

        while (atual != root) {
            atual = atual.getParent();
            profundidade++;
        }

        return profundidade;
    }

    public void insert(int elemento) {

        No novo = new No(elemento);

        if (root == null) {
            root = novo;
            size++;
            return;
        }

        No atual = root;
        No pai = null;

        while (atual != null) {

            pai = atual;

            if (elemento < (int) atual.getElement()) {
                atual = atual.getChildrenEsq();
            } else {
                atual = atual.getChildrenDir();
            }
        }

        novo.setParent(pai);

        if (elemento < (int) pai.getElement()) {
            pai.setChildrenEsq(novo);
        } else {
            pai.setChildrenDir(novo);
        }

        size++;
    }

    public No find(int elemento) throws InvalidPositionExceptionBinaria {

        if (isEmpty()) {
            throw new InvalidPositionExceptionBinaria("Árvore vazia");
        }

        No atual = root;

        while (atual != null) {

```

```

        if (elemento == (int) atual.getElement()) {
            return atual;
        }

        if (elemento < (int) atual.getElement()) {
            atual = atual.getChildrenEsq();
        } else {
            atual = atual.getChildrenDir();
        }
    }

    throw new InvalidPositionExceptionBinaria("Nó não encontrado");
}

public void remove(int elemento) throws InvalidPositionExceptionBinaria {

    if (isEmpty()) {
        throw new InvalidPositionExceptionBinaria("Árvore vazia");
    }

    No atual = root;
    No pai = null;

    while (atual != null && (int) atual.getElement() != elemento) {

        pai = atual;

        if (elemento < (int) atual.getElement()) {
            atual = atual.getChildrenEsq();
        } else {
            atual = atual.getChildrenDir();
        }
    }

    if (atual == null) {
        throw new InvalidPositionExceptionBinaria("Nó não encontrado");
    }

    if (atual.getChildrenEsq() == null && atual.getChildrenDir() == null) {

        if (atual == root) {
            root = null;
        } else if (pai.getChildrenEsq() == atual) {
            pai.setChildrenEsq(null);
        } else {
            pai.setChildrenDir(null);
        }
    }

    else if (atual.getChildrenEsq() == null) {

        if (atual == root) {

```

```

        root = atual.getChildrenDir();
    } else if (pai.getChildrenEsq() == atual) {
        pai.setChildrenEsq(atual.getChildrenDir());
    } else {
        pai.setChildrenDir(atual.getChildrenDir());
    }
}

else if (atual.getChildrenDir() == null) {

    if (atual == root) {
        root = atual.getChildrenEsq();
    } else if (pai.getChildrenEsq() == atual) {
        pai.setChildrenEsq(atual.getChildrenEsq());
    } else {
        pai.setChildrenDir(atual.getChildrenEsq());
    }
}

else {

    No sucessorPai = atual;
    No sucessor = atual.getChildrenDir();

    while (sucessor.getChildrenEsq() != null) {
        sucessorPai = sucessor;
        sucessor = sucessor.getChildrenEsq();
    }

    atual.setElement(sucessor.getElement());

    if (sucessorPai.getChildrenEsq() == sucessor) {
        sucessorPai.setChildrenEsq(sucessor.getChildrenDir());
    } else {
        sucessorPai.setChildrenDir(sucessor.getChildrenDir());
    }
}

size--;
}

public void mostrar() throws InvalidPositionExceptionBinaria {

    if (isEmpty()) {
        throw new InvalidPositionExceptionBinaria("Árvore vazia");
    }

    int altura = height(root) + 1;
    int largura = 80;

    String[][] matriz = new String[altura][largura];

    preencherMatriz(root, matriz, 0, largura / 2);
}

```

```

    for (int i = 0; i < altura; i++) {
        for (int j = 0; j < largura; j++) {

            if (matriz[i][j] == null) {
                System.out.print(" ");
            } else {
                System.out.print(matriz[i][j]);
            }
        }
        System.out.println();
    }
}

private void preencherMatriz(No node, String[] [] matriz, int linha, int coluna) {

    if (node == null) {
        return;
    }

    matriz[linha][coluna] = String.valueOf(node.getElement());

    preencherMatriz(node.getChildrenEsq(), matriz, linha + 1, coluna - 10);
    preencherMatriz(node.getChildrenDir(), matriz, linha + 1, coluna + 10);
}
}

```

Estrutura do Nó

```

public class No {

    private No parent;
    private No childrenEsq;
    private No childrenDir;
    private Object elemento;

    public No(Object elemento) {
        this.elemento = elemento;
        this.parent = null;
        this.childrenEsq = null;
        this.childrenDir = null;
    }

    public Object getElement() {
        return elemento;
    }

    public void setElement(Object elemento) {
        this.elemento = elemento;
    }

    public No getParent() {
        return parent;
    }
}

```



```

public void setParent(No parent) {
    this.parent = parent;
}

public No getChildrenEsq() {
    return childrenEsq;
}

public void setChildrenEsq(No childrenEsq) {
    this.childrenEsq = childrenEsq;
}

public No getChildrenDir() {
    return childrenDir;
}

public void setChildrenDir(No childrenDir) {
    this.childrenDir = childrenDir;
}
}

```

Interface Position

```

public interface Position {

    int size();
    boolean isEmpty();

    No getRoot();

    No parent(No node);
    No leftChild(No node);
    No rightChild(No node);

    boolean hasLeft(No node);
    boolean hasRight(No node);

    boolean isRoot(No node);
    boolean isInternal(No node);
    boolean isExternal(No node);

    boolean isLeftChild(No node);
    boolean isRightChild(No node);

    int height(No node);
    int depth(No node);

    void insert(int elemento);
    No find(int elemento);
    void remove(int elemento);

    void mostrar();
}

```

